

Unified Education Interface — Prisma ORM Layer

Go 1 (Week 4): Project Setup & schema.prisma

Repository	uei-backend
Commit	22e30be — “Go 1 (Week 4): Prisma project setup + schema.prisma for all 14 tables”
Author	Kashish Soni (finalyearproject.uei.27@gmail.com)
Followed by	Go 2 — baseline migration + repository/query layer (documented separately)
Date	September 24, 2026

1. What This Submission Covers

This was the first submission on the UEI (Unified Education Interface) Student Analytics & Placement Readiness backend: standing up the Prisma project itself and translating the already-designed, already-running PostgreSQL database (schema_v2.sql — 14 tables, 8 enums) into an idiomatic Prisma schema, so every later query, migration and API in the project can be built on top of a type-safe, verified data layer instead of raw SQL.

- **prisma/schema.prisma** — all 14 tables and 8 enums, hand-translated field by field into camelCase Prisma models, each mapped back to its real snake_case column/table name with @map / @@map so the database itself stays untouched.
- **Project scaffold** — package.json, tsconfig.json, .env.example, .gitignore — a standard Prisma + TypeScript setup.
- **src/test-connection.ts** — a runnable script that connects via Prisma Client and runs one relational query, to prove the schema is wired up correctly against a live database.
- **Verification** — every column in the schema was cross-checked, field by field, against a live PostgreSQL database that schema_v2.sql was actually run on: zero missing or mismatched columns.

2. schema.prisma — Translating schema_v2.sql

Approach. Rather than running `prisma db pull` (which would have produced a raw, non-idiomatic introspection dump with snake_case field names), the schema was hand-written to match the already-running database, so that field names could be made idiomatic (camelCase) while every real column/table/type name is preserved underneath via @map / @@map. For example:

```
model StudentProfile {
  id          String @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
  rollNumber  String  @unique @map("roll_number") @db.VarChar(50)
  aadharNo    String? @unique @map("aadhar_no") @db.VarChar(20)
  aparId      String? @unique @map("apar_id") @db.VarChar(50)
  aisheId     String? @map("aishe_id") @db.VarChar(50)
  ...
  @@index([section], map: "idx_students_section")
  @@map("student_profiles")
}
```

Every foreign key, every UNIQUE constraint and every CREATE INDEX in schema_v2.sql was carried over — including the canonical-ID fields the UEI project design calls for (roll number, registration number, Aadhar

number, APAR ID, AISHE ID) so students and institutions can be tracked consistently across the platform.

2.1 The 8 Enums

Enum	Purpose
UserRole	Login role: student, faculty, HOD, etc.
CourseType	Theory / lab / elective classification for a course.
AttendanceStatus	Present / absent / leave states used by attendance records.
InternshipStatus	Ongoing / completed / upcoming state of a student's internship.
AchievementCategory	Type of achievement being recorded (academic, sports, co-curricular, etc.).
RiskLevel	Low / medium / high risk classification used by the weak-student radar.
SkillProficiency	Beginner / intermediate / advanced level for a recorded student skill.
PlacementOutcome	Placement status outcome (placed, in-process, not placed, etc.).

2.2 The 14 Models

Model	What it represents
User	Master auth table — email, password hash, role, active flag.
StudentProfile	The student “Digital Twin” — canonical IDs (roll no., registration no., Aadhar, APAR, AISHE), CGPA, attendance, section, profile links.
FacultyProfile	Faculty member record — employee ID, department, designation.
Course	Course catalog — code, title, semester, credits, type.
CourseEnrollment	M:N bridge between students and courses (a student's enrollment in a course).
PlacementReadiness	A student's placement status, readiness score and package.
WeakStudentRadar	AI/ML-driven risk predictions per student/course, with risk factor and score.
AttendanceRecord	Per-session attendance status for a student's course enrollment.
StudentSkill	Skills recorded against a student, with proficiency level.
Internship	A student's internship record — dates, status, details.
Achievement	Achievements recorded against a student, categorized.
SemesterRecord	Per-semester SGPA history for a student.
FacultyCourseAssignment	Which faculty member teaches which course/section, and when.
AuditLog	System-wide action trail, tied back to the User who performed it.

3. Connection Test — src/test-connection.ts

A small runnable script proving the schema is wired up correctly end to end: it connects via Prisma Client, counts rows in four core tables (users, student profiles, faculty profiles, courses), then runs one relational query — a student with their user account and first two course enrollments (with course details) included — to confirm the model relations resolve correctly against the live database.

```
npm run test:connection
```

4. Verification Performed

Every column in `schema.prisma` was cross-checked, field by field, against a live PostgreSQL database that `schema_v2.sql` was actually run on — zero missing or mismatched columns. Because the schema was hand-written rather than produced by `prisma db pull`, this field-by-field check was the substitute for introspection: it confirms that running `npx prisma db pull --print` against the same database would show it lining up 1:1 with this file.

5. How to Run This Submission

```
# 1. Install dependencies
npm install

# 2. Point Prisma at your own database
cp .env.example .env
# edit .env and set DATABASE_URL to your Postgres connection string

# 3. Make sure schema_v2.sql has already been run against that database
# (creates the 14 tables/enums this schema.prisma describes)

# 4. Validate the schema and generate the Prisma Client
npx prisma validate
npx prisma generate

# 5. Sanity-check the connection
npm run test:connection
```

6. Files Changed This Submission

File	Diff	Purpose
<code>.env.example</code>	+7	Template for DATABASE_URL (new)
<code>.gitignore</code>	+5	Keeps real .env / node_modules / dist out of git (new)
<code>README.md</code>	+71	Project overview + Go 1 write-up (new)
<code>package.json</code>	+25	Prisma + TypeScript project scaffold (new)
<code>prisma/schema.prisma</code>	+400	All 14 models, 8 enums, FKs, unique constraints, indexes (new)
<code>src/test-connection.ts</code>	+43	Prisma Client connection/relation smoke test (new)
<code>tsconfig.json</code>	+15	TypeScript compiler configuration (new)

7 files changed, 566 insertions(+) — commit 22e30be.

7. Known Schema Note

Most foreign-key columns (e.g. `course_enrollments.student_id`) are not `NOT NULL` in `schema_v2.sql`, so their Prisma fields are modeled as optional to stay byte-for-byte accurate to the real database, rather than silently tightening an already-reviewed schema. Flagged as a reasonable candidate for a future migration if the team wants to enforce it at the DB level.

8. What Followed

The next submission (Go 2) fixed an enum-mapping bug found in this schema, baselined the existing database into a formal Prisma migration history, and added a first repository/service layer with runnable queries for the student, risk, faculty and analytics workflows — documented separately.

UEI — Unified Education Interface · Prisma ORM layer over schema_v2.sql (14 tables)